

Version 2.0

January 2026



OPERATING SYSTEMS PRACTICUM

MODULE 6 - THREAD AND MULTITHREAD

Author:

Ir. Denar Regata Akbi, S.Kom., M.Kom.

Putri Nayla Sabri

Muhammad Zamzam Baihaqi

INFORMATICS LABORATORY

University of Muhammadiyah Malang

TABLE OF CONTENTS

TABLE OF CONTENTS.....	2
INTRODUCTION.....	2
OBJECTIVES.....	3
MODULE TARGETS.....	3
PREPARATION.....	3
KEYWORDS.....	3
THEORETICAL BACKGROUND.....	4
1. WHAT IS A THREAD AND WHAT IS MULTITHREADING?.....	4
1.1. Thread as a lightweight process.....	4
1.2. Single-threaded vs Multithreaded programs.....	5
1.3. Thread vs Process: key differences.....	5
1.4. Why use Multithreading? Benefits in real applications.....	6
2. THREAD MODELS IN OPERATING SYSTEMS.....	6
2.1. User-Level Threads (ULT).....	7
2.2. Kernel-Level Threads (KLT).....	7
2.3. Hybrid Model and Linux Implementation.....	8
3. MULTITHREADING IN PRACTICE ON ROCKY LINUX.....	9
3.1. POSIX Threads (pthreads): basic structure.....	10
EXERCISE 1.....	11
3.2. Creating and managing threads with 'pthread_create()'.....	11
EXERCISE 2.....	12
3.3. Observing threads in the system ('ps -T', 'htop').....	12
EXERCISE 3.....	13
EXERCISE 4.....	13
4. COMMON CONCURRENCY ISSUES AND SAFE PRACTICES.....	13
4.1. Race conditions: when threads interfere.....	13
EXERCISE 5.....	14
4.2. Basic mutual exclusion with mutex ('pthread_mutex_t').....	15
4.3. Best practices: avoiding forceful termination and deadlock.....	15
PRACTICAL APPLICATIONS.....	18
PRACTICE.....	18
ASSESSMENT RUBRIC.....	21
ASSESSMENT SCALE.....	21



INTRODUCTION

OBJECTIVES

1. Understand the concept of a thread as a lightweight unit of execution within a process.
2. Identify, create, and observe threads using CLI tools like ps -T and htop.
3. Apply basic synchronization techniques (mutex) to prevent race conditions in multithreaded programs.

MODULE TARGETS

1. Differentiate between single-threaded and multithreaded programs.
2. Compile and run a simple POSIX Threads (pthreads) program.
3. Monitor active threads in the system using ps -T and htop -H.
4. Implement mutual exclusion with pthread_mutex_t to protect shared data.

PREPARATION

1. Muslim students may recite the following prayer before starting the lesson
رَضِيتُ بِاللّهِ رَبًّا وَبِالْإِسْلَامِ دِينًا وَبِمُحَمَّدٍ نَبِيًّا وَرَسُولًا. رَبِّ زِدْنِي عِلْمًا وَارْزُقْنِي فَهْمًا
2. Laptop or personal computer
3. Rocky Linux
4. Virtual Box
5. Strong motivation and commitment to learning

KEYWORDS

Thread, Multithreading, Lightweight Process, pthread_create(), pthread_join(), pthread_mutex_t, Mutex, Race Condition, PID, TID, LWP, ps -T, htop -H, Shared Memory, Context Switch, One-to-One Model



THEORETICAL BACKGROUND

1. WHAT IS A THREAD AND WHAT IS MULTITHREADING?

In modern operating systems like Rocky Linux, programs rarely run as a single, linear sequence of instructions. Instead, they often perform multiple tasks at once—such as downloading data while updating a user interface or handling multiple client requests simultaneously. This capability is made possible through **threads** and **multithreading**.

A **thread** is the smallest unit of execution within a process. While a **process** represents a complete program with its own memory space, a **thread** shares that memory space with other threads in the same process—but maintains its own stack, program counter, and register set. This design allows threads to work concurrently while efficiently sharing resources.

Multithreading refers to the ability of a process to create and manage multiple threads that execute concurrently. It is a foundational concept in modern system programming, web servers, GUI applications, and real-time systems enabling responsiveness, performance, and efficient resource utilization.

This section clarifies what threads are, how they differ from processes, and why multithreading is essential in real-world software.

1.1. Thread as a lightweight process

A thread is often called a **lightweight process** because it behaves like a process but with significantly less overhead.

Unlike a full process, which requires its own memory space, file descriptors, and kernel resources a thread:

- Shares code, data, heap, and open files with other threads in the same process,
- Has its own **stack**, **register state**, and **thread ID (TID)**,
- Can be created and destroyed much faster than a process.

Core understanding:

Creating a new thread is typically **10–100x faster** than creating a new process because no memory duplication or resource reinitialization is needed.

In Linux, threads are implemented as **tasks** managed by the kernel using the same scheduling mechanism as processes—but grouped under a common **thread group ID (TGID)**, which equals the PID of the main thread.



You can view threads using:

```
praktikumso@10:~$ ps -T -p [PID]
```

or

```
praktikumso@10:~$ htop -H
```

1.2. Single-threaded vs Multithreaded programs

Single-Threaded Programs

- Execute one instruction at a time.
- If one task blocks (e.g., waiting for I/O), the entire program halts.
- Simple to write and debug—but inefficient for I/O-bound or interactive applications.

For example: a basic shell script that downloads a file, then processes it sequentially.

Multithreaded Programs

- Contain **two or more threads** running concurrently within the same process.
- While one thread waits for disk or network I/O, another can continue processing.
- Enable **responsiveness** (e.g., a GUI remains usable during background tasks) and **parallelism** (on multi-core CPUs).

For example: a web server handling 100 client requests simultaneously using one thread per request.

Note: Multithreading introduces complexity—especially when threads access shared data (see Section 4 on race conditions).

1.3. Thread vs Process: key differences

Feature	Process	Thread
Memory Space	Separate virtual address space	Shares memory with other threads in the same process
Creation Overhead	High (copy page tables, file descriptors, etc.)	Low (share most resources)
Communication	Requires IPC (pipes, sockets, shared memory)	Direct access to shared variables
Isolation	Failure in one process doesn't affect others	A crashing thread may terminate the entire process
Context Switch Cost	Higher (full memory context switch)	Lower (same memory space)
Resource Usage	More CPU and memory	Lightweight and efficient



Rule of Thumb:

Use **multiple processes** when you need strong isolation (e.g., security, fault tolerance).

Use **multiple threads** when you need performance and shared data access within a single application.

1.4. Why use Multithreading? Benefits in real applications

Multithreading is not just a theoretical concept—it powers real-world systems:

1. Improved Responsiveness

- In GUI applications (e.g., text editors, browsers), the main thread handles user input, while background threads perform saving, spell-checking, or rendering.
- Users never see a “frozen” interface.

2. Resource Sharing

- Threads in the same process share code, data, and open files—eliminating the need for complex inter-process communication (IPC).

3. Economy

- Thread creation, switching, and termination are cheaper than processes—critical for high-performance servers.

4. Scalability on Multi-Core Systems

- On modern CPUs with multiple cores, threads can run in parallel—not just concurrently—dramatically speeding up computation-heavy tasks (e.g., video encoding, scientific simulation).

Real-World Examples:

- **Web Servers** (Apache, Nginx): One thread per client connection.
- **Database Systems**: Separate threads for query parsing, I/O, and logging.
- **Media Players**: One thread decodes audio, another renders video, a third handles user controls.

Summary:

Threads enable efficient, responsive, and scalable software by allowing multiple flows of execution within a single process. Understanding the difference between threads and processes and knowing when to use multithreading is essential for system programming, server development, and performance optimization in Linux environments like Rocky Linux.

2. THREAD MODELS IN OPERATING SYSTEMS

While a thread is the basic unit of execution within a process, how threads are **managed and scheduled** depends on the underlying **thread model** used by the operating system. Different models offer trade-offs between performance, portability, and kernel involvement.

Understanding these models helps explain why multithreaded programs behave differently across systems and why Linux uses a specific hybrid approach.



This section introduces the three primary thread models: **User-Level Threads (ULT)**, **Kernel-Level Threads (KLT)**, and the **Hybrid (Many-to-Many) Model**, with emphasis on how Linux implements threading in practice.

2.1. User-Level Threads (ULT)

A. What Are User-Level Threads?

User-Level Threads (ULTs) are managed entirely by a **user-space thread library** (e.g., GNU Portable Threads). The kernel is **unaware** of their existence; it sees only a single process, even if that process contains dozens of threads.

How It Works

- Thread creation, scheduling, and context switching happen in user space.
- No system calls are needed for thread operations → very fast context switches.
- A single kernel thread (process) supports all ULTs.

Advantages

- **High performance:** Thread operations avoid kernel overhead.
- **Portability:** Can run on any OS even those without native thread support.
- **Custom scheduling:** Applications can define their own scheduling policies.

Disadvantages

- **Blocking I/O blocks all threads:** If one ULT performs a blocking system call (`read()`), the entire process (and all its threads) is blocked.
- **No true parallelism:** On multi-core systems, ULTs cannot run simultaneously because the kernel schedules only one process at a time.

Example: Early versions of Java used ULTs before adopting native threads.

2.2. Kernel-Level Threads (KLT)

A. What Are Kernel-Level Threads?

Kernel-Level Threads (KLTs) are managed directly by the **operating system kernel**. Each thread is a schedulable entity known to the kernel, just like a process.

How It Works

- Thread creation, scheduling, and synchronization are handled by the kernel.
- Each thread has its own **kernel thread ID (TID)** and appears in system tools like `ps` and `htop`.
- The kernel can schedule multiple threads from the same process on different CPU cores.



Advantages

- **True concurrency:** Multiple threads can run in parallel on multi-core systems.
- **Robustness:** If one thread blocks (e.g., on I/O), others continue executing.
- **Integration with OS services:** Threads benefit from kernel-level scheduling, signals, and debugging.

Disadvantages

- **Higher overhead:** Thread operations require system calls and kernel involvement.
- **Less portable:** Requires OS support for threading.

Linux uses KLTs as the foundation for its threading model (via the 'clone()' system call).

2.3. Hybrid Model and Linux Implementation

A. The Hybrid (Many-to-Many) Model

The **Hybrid Model** combines ULTs and KLTs:

- Multiple user-level threads are mapped to a smaller or equal number of kernel threads.
- The thread library handles ULT scheduling, while the kernel schedules KLTs.

This model aims to get the best of both worlds:

- Fast user-space switching,
- Plus true parallelism and non-blocking I/O.

However, it is **complex to implement** and has largely fallen out of favor in modern systems.

B. How Linux Implements Threading: One-to-One Model

Contrary to older textbooks, **modern Linux does not use the hybrid model**. Instead, it uses a **one-to-one model**:

- **Each user thread = one kernel thread.**
- Implemented via the '`clone()`' system call, which creates a new task that shares memory, file descriptors, and signal handlers with its parent—but has its own stack and registers.

This means:

- Every thread created by '`pthread_create()`' becomes a **separate schedulable entity** in the kernel.
- Tools like '`ps -T`' or '`htop -H`' show each thread as a separate line with its own TID.



C. Verification in Rocky Linux:

Compile a simple pthread program (see Section 5)

```
praktikumso@localhost:~$ gcc -o thread_demo.c -lpthread
```

Run it in background

```
praktikumso@localhost:~$ ./thread_demo &
```

View threads

```
praktikumso@localhost:~$ ps -T -p $(pgrep thread_demo)
```

Output will show multiple TIDs under the same PID—confirming one-to-one mapping.

D. Why This Matters for Students

You don't need to manage thread models manually **POSIX threads ('pthreads') abstract this.**

- But understanding that **each thread is a real kernel entity** explains:
 - Why 'htop' shows many threads,
 - Why killing a thread's TID terminates part of a process,
 - Why thread creation has measurable overhead.

Summary :

Thread models define how threads are scheduled and managed. While ULTs offer speed and KLTs offer true concurrency, **Linux uses a one-to-one kernel-level model** via 'pthreads' and 'clone()'. This gives you both performance and parallelism—making it ideal for servers, responsive applications, and real-time systems.

3. MULTITHREADING IN PRACTICE ON ROCKY LINUX

While the concept of threads is theoretical, their implementation in real systems is concrete and observable. In Linux-based systems like **Rocky Linux**, multithreading is primarily supported through the **POSIX Threads (pthreads)** API—a standardized C library for creating and managing threads. This section bridges theory and practice by introducing the basic structure of a pthread program, demonstrating thread creation, and showing how to observe threads directly in the terminal.

All examples can be compiled and run on Rocky Linux using 'gcc' and standard system tools—making them ideal for hands-on lab sessions.



3.1. POSIX Threads (pthreads): basic structure

A. What Is POSIX Threads (pthreads)?

POSIX Threads (pthreads) are a standardized C-language API defined by the IEEE POSIX 1003.1c standard. It provides functions for:

- Creating threads,
- Synchronizing threads (using mutexes, condition variables, etc.),
- Managing thread attributes.

In Rocky Linux, pthreads are implemented as part of the **glibc** (GNU C Library) and use the **one-to-one kernel-level threading model** (each user thread = one kernel task).

B. Basic Structure of a pthread Program

A minimal multithreaded C program includes:

1. Include '`<pthread.h>`'
2. Define a thread function (the code each thread will run)
3. Use '`pthread_create()`' to spawn threads
4. Use '`pthread_join()`' to wait for threads to finish

Key Rule:

Every thread starts execution at the same function but with its own stack and register state.

Example skeleton:

```

1 #include <pthread.h>
2 #include <stdio.h>
3
4 //thread function
5 void* thread_function(void* arg) {
6     printf("Hello from a thread!\n");
7     return NULL;
8 }
9
10 int main() {
11     pthread_t tid;
12
13     //create thread
14     pthread_create(&tid, NULL, thread_function, NULL);
15
16     //Wait for thread to finish
17     pthread_join(tid, NULL);
18
19     printf("Thread complete.\n");
20     return 0;
21 }
```



Quick tips: how to make C program file:

```
praktikumso@10:~$ vim thread_demo.c
```

To compile on Rocky Linux:

```
praktikumso@10:~$ gcc -o thread_demo thread_demo.c -lpthread
```

Note: The '-lpthread' flag links the pthread library.

EXERCISE 1

Create a simple C program that spawns one thread using `pthread_create()`, then compile and run it on Rocky Linux.

3.2. Creating and managing threads with 'pthread_create()'

A. The 'pthread_create()' Function

Syntax:

```
24 int pthread_create(
25     pthread_t *thread,
26     const pthread_attr_t *attr,
27     void *(*start_routine)(void*)
28     void *arg
29 );
```

B. Passing Data to Threads

Threads can receive arguments via the '`void* arg`' parameter. This allows customization per thread.

Example: Passing an integer ID:

```
32 void* worker(void* arg){
33     int id = *(int*)arg;
34     printf("Worker %d is running\n", id);
35     return NULL;
36 }
37
38 int main() {
39     pthread_t t1, t2;
40     int id1 = 1, id2 = 2;
41
42     pthread_create(&t1, NULL, worker, &id1);
43     pthread_create(&t2, NULL, worker, &id2);
44
45     pthread_join(t1, NULL);
46     pthread_join(t2, NULL);
47
48     return 0;
49 }
```



Caution: Avoid passing the loop variable directly in loops—use separate storage (e.g., array of IDs) to prevent race conditions on the variable itself.

Thread Lifecycle

- A thread begins when `'pthread_create()'` is called.
- It ends when the thread function returns or `'pthread_exit()'` is called.
- The main thread should join all child threads using `'pthread_join()'` to avoid premature termination.

Best Practice: Always call `'pthread_join()'` unless you explicitly detach a thread with `'pthread_detach()'`.

EXERCISE 2

Modify the previous program (`'pthread_create()'`) so the thread accepts an integer ID as an argument and prints it to the terminal.

3.3. Observing threads in the system ('ps -T', 'htop')

Unlike processes, threads share the same PID—but each has a unique Thread ID (TID). Rocky Linux exposes these TIDs through standard tools.

Using 'ps -T' to View Threads

The `'-T'` flag shows all threads within a process:

```
praktikumso@10:~$ ./thread_demo &
praktikumso@10:~$ ps -T -p $(pgrep thread_demo)
```

Sample output:

PID	SPID	TTY	TIME	CMD
1234	1234	pts/0	0:00	thread_demo
1234	1235	pts/0	0:00	thread_demo
1234	1236	pts/0	0:00	thread_demo

- **PID:** Process ID (same for all threads)
- **SPID:** Thread ID (also called LWP – Light Weight Process)

Insight: All threads share the same PID but have different SPIDs—confirming the one-to-one kernel model.

Using 'htop' to Monitor Threads

Run `'htop'`, then press `H` (capital H) to toggle thread view.



In thread mode:

- Each thread appears as a separate line.
- The main thread is usually listed first.
- CPU and memory usage are shown per thread.

Why This Matters:

Observing threads helps debug performance issues (e.g., one thread consuming 100% CPU) or verify that multithreading is working as expected.

Summary:

Multithreading in Rocky Linux is practical, observable, and essential for modern system programming. By using `'pthread'`, you can create concurrent programs that leverage multiple CPU cores. Tools like `'ps -T'` and `'htop'` let you verify thread behavior directly in the terminal—critical for debugging and performance tuning.

EXERCISE 3

Run your multithreaded program in the background, then use `ps -T` to identify the PID and TID (Thread ID) of each thread.

EXERCISE 4

Install `htop` (if not already present), run your multithreaded program, and press H in `htop` to view all threads as separate entries.

4. COMMON CONCURRENCY ISSUES AND SAFE PRACTICES

Multithreading enables powerful performance and responsiveness—but it also introduces new challenges when multiple threads access shared data simultaneously. Without proper coordination, threads can interfere with each other, leading to unpredictable behavior, data corruption, or system instability.

This section explains the most common concurrency problem—race conditions—and introduces the simplest and most widely used solution: mutual exclusion with mutexes. It also outlines best practices to avoid dangerous patterns like forced termination and deadlock.

4.1. Race conditions: when threads interfere

A. What Is a Race Condition?

A **race condition** occurs when two or more threads access shared data **concurrently**, and at least one of them **modifies** the data, without proper synchronization. The final result depends on the timing or **order** of execution—which is unpredictable.

Core Understanding:



A race condition does **not** mean threads are “racing” to finish—it means the program’s correctness **depends on timing**, which is unreliable.

Example: Counter Increment

Consider this simple C code executed by two threads:

```
51 int counter = 0;
52
53 void* increment(void* arg){
54     for(int i = 0; i < 100000; i++){
55         counter++;
56     }
57     return NULL;
58 }
```

If two threads run 'increment()' concurrently, the final value of 'counter' may be **less than 200,000** because the 'counter++' operation involves:

1. Read current value,
2. Add 1,
3. Write back.

If both threads read the same value before either writes, one update is lost.

Real Impact:

In banking systems, race conditions could cause incorrect balances. In embedded systems, they might trigger safety failures.

B. How to Detect Race Conditions?

- **Symptoms:** Intermittent bugs, inconsistent output, crashes under load.
- **Tools:** Use 'helgrind' (part of Valgrind) to detect data races:

```
praktikumso@10:~$ gcc -g -o race race.c -lpthread
```

```
praktikumso@10:~$ valgrind --tool=helgrind ./race
```

Note: Race conditions often disappear during debugging—making them hard to reproduce.

EXERCISE 5

Write a C program where **two threads access a shared global variable** without a mutex, then observe inconsistent output by running the program multiple times.



4.2. Basic mutual exclusion with mutex ('pthread_mutex_t')

A. What Is a Mutex?

A **mutex** (short for mutual exclusion) is a synchronization object that ensures only **one thread** can access a critical section of code at a time.

In POSIX threads ('**pthread**s'), a mutex is declared as:

```
61 pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
```

Using a Mutex: Step-by-Step

1. **Declare** the mutex (global or shared).
2. **Lock** before entering the critical section.
3. **Unlock** after leaving it.

Example (fixing the counter):

```
1 #include <pthread.h>
2 #include <stdio.h>
3
4 int counter = 0;
5 pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
6
7 void* increment(void* arg){
8     for(int i = 0; i < 100000; i++){
9         pthread_mutex_lock(&lock);
10        counter++;
11        pthread_mutex_unlock(&lock);
12    }
13    return NULL;
14 }
15
```

Result: Final '**counter**' will always be **200,000**.

Important Rules

- **Always unlock** what you lock—otherwise, other threads will **block forever**.
- **Never assume order:** Threads may run in any sequence.
- **Keep critical sections small:** Holding a lock too long reduces concurrency.

Observation in Rocky Linux:

While one thread holds the mutex, others calling '**pthread_mutex_lock()**' will **block** (sleep) until the lock is released—visible in '**htop**' as reduced CPU usage.

4.3. Best practices: avoiding forceful termination and deadlock

A. Avoid Forceful Thread Termination



Never use '`pthread_cancel()`' or kill threads abruptly unless absolutely necessary. Why?

- **Resources may leak:** Open files, memory, or locks won't be cleaned up.
- **Mutexes may remain locked:** Causing other threads to hang indefinitely.

Safe Alternative: Use a **flag variable** to signal a thread to exit gracefully:

```
1 #include <pthread.h>
2 #include <stdio.h>
3
4 volatile int stop_flag = 0;
5
6 void* worker(void* arg){
7     while (!stop_flag){
8         //DO work
9     }
10    // Clean up resource
11    return NULL
12 }
13
14 // To stop:
15 stop_flag = 1;
16 pthread_join(thread_id, NULL); //wait for clean exit
17
```

Note: this pattern is used in real-world servers and daemons.

B. Avoid Deadlock

Deadlock occurs when two or more threads are **waiting for each other** to release resources, causing all to hang forever.

Classic Deadlock Scenario:

- Thread A locks mutex X, then tries to lock mutex Y.
- Thread B locks mutex Y, then tries to lock mutex X.
- Both wait indefinitely.

Prevention Strategies:

1. **Lock Ordering:** Always acquire mutexes in a **fixed global order** (e.g., always lock X before Y).
2. **Avoid Nested Locks:** If possible, design code to use only one mutex at a time.
3. **Use Timeouts** (advanced): '`pthread_mutex_timedlock()`' to avoid infinite waits.

Rule of Thumb:

If you need more than one mutex, **document the locking order clearly**.



Summary of Safe Practices

Practice	Why It Matters
Protect shared data with mutexes	Prevents race conditions
Keep critical sections short	Maximizes concurrency
Never kill threads abruptly	Avoids resource leaks and broken state
Use consistent lock ordering	Prevents deadlock
Test under load	Race conditions often appear only with high concurrency

Summary:

Concurrency is powerful but requires discipline. By understanding race conditions and using mutexes correctly, you can write multithreaded programs that are both fast and correct. These skills are essential for building responsive applications, web servers, and real-time systems in Linux environments.



PRACTICAL APPLICATIONS

PRACTICE

Objective: Students make program C thread and simulate it on terminal.
(The exercise is not shown yet; students wait for instructions during the practice session.)

DEMO – CASE STUDY

The Situation

A campus IoT team manages environmental sensor nodes running on Rocky Linux. Each node uses a multithreaded data collector to process temperature readings. During stress testing, the team reports inconsistent readings—some values duplicate while others disappear entirely. The lead engineer suspects a concurrency issue but needs immediate diagnosis. The problematic code exists in two versions:

counter_race.c - without synchronization (vulnerable to race conditions) contain:

```
1 #include <pthread.h>
2 #include <stdio.h>
3 #define ITER 100000
4 int counter = 0;
5 void* worker(void* arg) {
6     for (int i = 0; i < ITER; i++) counter++;
7     return NULL;
8 }
9 int main() {
10     pthread_t t1, t2;
11     pthread_create(&t1, NULL, worker, NULL);
12     pthread_create(&t2, NULL, worker, NULL);
13     pthread_join(t1, NULL);
14     pthread_join(t2, NULL);
15     printf("Final: %d\n", counter);
16     return 0;
17 }
18
```

counter_mutex.c - with mutex protection (thread-safe) contain:

```
1 #include <pthread.h>
2 #include <stdio.h>
3 #include <stdlib.h>
4 #include <time.h>
5
6 #define DEFAULT_ITERATIONS 100000
7
8 int counter = 0;
9 pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
10 int num_threads = 2;
11 int iterations = DEFAULT_ITERATIONS;
12
13 void* worker(void* arg) {
14     // Tambahkan kode untuk menghitung counter dengan sinkronisasi
15     for (int i = 0; i < iterations; i++) {
16         pthread_mutex_lock(&lock);
17         counter++;
18         pthread_mutex_unlock(&lock);
19     }
20     return NULL;
21 }
22
```



```

24 int main(int argc, char* argv[]) {
25     // Tambahkan kode untuk:
26     // 1. Validasi argumen baris perintah
27     // 2. Inisialisasi barrier
28     // 3. Pengukuran waktu eksekusi per thread
29     // 4. Penanganan sumber daya
30
31     // Buat dan jalankan thread
32     pthread_t* threads = malloc(num_threads * sizeof(pthread_t));
33     if (!threads) {
34         perror("malloc failed");
35         return 1;
36     }
37
38     for (int i = 0; i < num_threads; i++) {
39         if (pthread_create(&threads[i], NULL, worker, NULL) != 0) {
40             perror("pthread_create failed");
41             // Cleanup: hentikan semua thread yang sudah dibuat
42             for (int j = 0; j < i; j++) {
43                 pthread_cancel(threads[j]);
44             }
45             free(threads);
46             return 1;
47         }
48     }
49
50     for (int i = 0; i < num_threads; i++) {
51         pthread_join(threads[i], NULL);
52     }
53
54     printf("\nFinal counter: %d\n", counter);
55     printf("Expected: %d\n", num_threads * iterations);
56
57     pthread_mutex_destroy(&lock);
58
59     free(threads);
60
61     return 0;
62 }

```

Note: The C source files are pre-prepared in `~/demo6/`. To build **muscle memory** and **technical fluency**, you will manually type and compile these scripts. This exercise is designed to train you in precision syntax handling, familiarize you with the C-threading library, and ensure you can navigate the Linux development environment effectively under pressure. The C source files have been prepared in advance. During the demo, you will only **compile, execute, observe, modify, and verify** not write code from scratch.

Your task is to **compile, run, observe, and explain** the behavior of both programs using only terminal commands demonstrating your understanding of threads, concurrency, and safe practices.

The Problems to Identify

As the system reliability engineer, you must diagnose these critical issues within 20 minutes:

- **Race Condition:** The unsynchronized counter produces inconsistent results (e.g., 198,432 instead of 200,000) due to read-modify-write hazards
- **Thread Visibility:** System tools must confirm multiple threads executing under a single process



- **Synchronization Validation:** The mutex-protected version must consistently output 200,000 across all runs
- **Performance Tradeoff:** Mutex implementation introduces measurable overhead that must be quantified

DEMO TASKS (Student Must Perform Live in Terminal)

Your task is to diagnose the concurrency issue and validate the solution through live terminal execution. You will be successful if:

1. Race Condition Verification

- Compile and run `counter_race.c` 3 times
- Document inconsistent counter values (e.g., 198,432 → 199,105 → 197,888)
- Verify thread count using `ps -T -p [PID]` and `htop` (press H)

2. Mutex Solution Validation

- Compile and run `counter_mutex.c` 3 times
- Confirm consistent output of **200,000** in all executions
- Measure performance impact using `time ./counter_race` vs `time ./counter_mutex`

3. Design Modification

Modify `counter_mutex.c` to implement a production-grade thread management system that:

- Command-Line Argument Handling: Accept dynamic thread count as first argument (e.g., `./counter_mutex 8`)
- Input Validation: Verify argument is integer between 1-100, return proper error message if invalid
- Thread Barrier Implementation: Add a pthread barrier to ensure all threads start simultaneously
- Per-Thread Performance Monitoring:
 - Record start/end time for each thread
 - Calculate and print individual thread execution times
 - Display average thread execution time
- Resource Cleanup Verification:
 - Properly destroy all mutexes and barriers
 - Add verification step to confirm no resource leaks
- Critical Section Optimization:
 - Reduce mutex lock scope to only the critical section
 - Explain why this optimization improves performance



Verification Requirements:

- Demonstrate the program works with 4 threads (output = 400,000)
- Show proper error handling with invalid inputs (e.g., `./counter_mutex abc`)
- Display per-thread execution times and average
- Confirm all resources are properly cleaned up

4. Technical Explanation

- Explain why race conditions occur in `counter_race.c`
- Describe how mutex prevents data corruption

Articulate the performance-correctness tradeoff observed

Demo Rules

1. No External Resources

Students must not open module PDFs, notes, or internet during demo.

2. Pre-Prepared Files

Only `counter_race.c` and `counter_mutex.c` are provided in `~/demo6/`.

No code writing—only compilation, execution, and observation.

3. Verification Requirement

All claims must be backed by live terminal output:

- Inconsistent vs. consistent counter values
- Thread listing via `ps -T`
- Timing comparison via `time`

4. Assessment Focus

- The assessor will evaluate:
- Correct use of system tools to view threads
- Understanding of race conditions vs. mutex protection
- Ability to articulate synchronization tradeoffs

5. Interactive Response

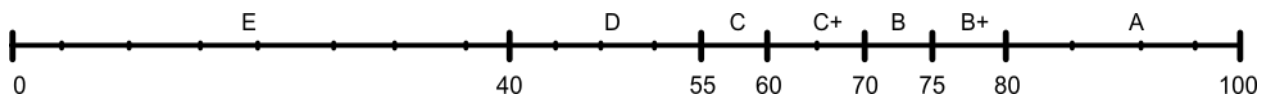
- Students must respond to prompts like:
- "Show me the TIDs for this process"
- "Explain why the race condition occurs"
- "What happens if we forget `pthread_mutex_unlock()`?"



ASSESSMENT RUBRIC

Assessment Aspect	Points (Total 100%)
EXERCISE	Total 5%
Exercise 1	1%
Exercise 2	1%
Exercise 3	1%
Exercise 4	1%
Exercise 5	1%
PRACTICE	Total 20%
Practice 1	10%
Practice 2	10%
DEMO	Total 75%
Case Study Completion	35%
Understanding Race Conditions & Diagnosis	10%
Understanding Mutexes & Synchronization	10%
Design Modification Skills	10%
Performance-Correctness Tradeoff Analysis	10%

ASSESSMENT SCALE



- A** = (81 - 100) → Sepuh
- B+** = (75 - 80) → Very Good
- B** = (70 - 74) → Good
- C+** = (60 - 69) → Fairly Good
- C** = (55 - 59) → Fair
- D** = (41 - 54) → Poor
- E** = (0 - 40) → Bro really...

